ITRI625 PROJECT

Theory Report — Deep Learning Intrusion Detection

Student name: Malonje Zimba **Student number**: 56507267

Student name: Precious Masetla **Student number**: 45645892

GitHub link: https://github.com/ITRI625-ML-Project/itri625-intrusion-detection

Table of Contents

1. Model Scenario 2
2. Model Architecture 2
2.1 Why a Multi-Layer Perceptron (MLP)? 2
2.2 Architecture summary 2
2.3 Design choices and justification 3
3. Model Result Analysis 3
3.1 Reading the training curves 4
3.2 Precision, recall, F1 and the confusion matrix 4
3.3 ROC curve, PR curve and AUC-ROC 5
3.4 Overall effectiveness 6
4. Real-World Use of Models in Cybersecurity 6
4.1 Strengths in practical deployments 6
4.2 Limitations and common challenges 6
4.3 References 7

1. Model Scenario

The model developed for this project is a binary intrusion-detection classifier. Given the recorded features of a single network/login session — packet size, protocol type, number of login attempts, session duration, whether encryption was used, the IP address's reputation score, the number of failed logins, browser type, and whether access occurred at an unusual time — the model predicts whether that session represents a genuine attack or normal user activity (the attack_detected label).

In a real security operations context, this kind of model would sit behind a monitoring layer that logs session metadata as it happens. Rather than requiring a security analyst to manually inspect every session, the model scores each session in real time and raises an alert only for sessions it flags as likely attacks. This is directly useful for intrusion detection: it reduces the volume of traffic a human analyst has to review, catches patterns (e.g. an unusual combination of many failed logins, a poor IP reputation score, and access at an odd hour) that are easy for a rule-based system to miss but straightforward for a trained model to learn, and can be deployed as a lightweight service (the FastAPI + Tkinter application built alongside this notebook) that other tools or a SOC dashboard could call.

2. Model Architecture

2.1 Why a Multi-Layer Perceptron (MLP)?

The dataset is plain tabular data: each row is one independent session described by a fixed set of numeric and categorical features, with no spatial structure (as an image would have, which is what Convolutional Neural Networks are designed to exploit) and no sequential/temporal ordering between rows (as a time series or a sentence would have, which is what Recurrent Neural Networks and LSTMs are designed to exploit). For this kind of fixed-length, unordered feature vector, a fully-connected feed-forward network (an MLP) is the natural and well-suited choice: every input feature can influence every hidden unit directly, which is exactly the flexibility needed to learn interactions between, for example, failed_logins and unusual_time_access without assuming any particular ordering or locality among the features.

2.2 Architecture summary

Layer	**Size / rate**	**Purpose**
Input	n_features (17 after one-hot encoding + scaling)	Receives the pre-processed session vector.
Dense	64 units	Uses ReLU activation function. Learns non-linear combinations of the input features.
BatchNormalization	64 units	Stabilises and speeds up training by normalising activations.
Dropout	rate = 0.3	Randomly disables 30% of units during training to reduce overfitting.
Dense	32 units	Uses ReLU activation function. Further compresses the representation.
BatchNormalization	32 units	As above.

Dropout	rate = 0.2	As above, lighter rate since the layer is smaller.
Dense	16 units	Final non-linear feature combination before classification.
Dense (output)	1 unit	Outputs P(attack_detected = 1) as a value between 0 and 1.

2.3 Design choices and justification

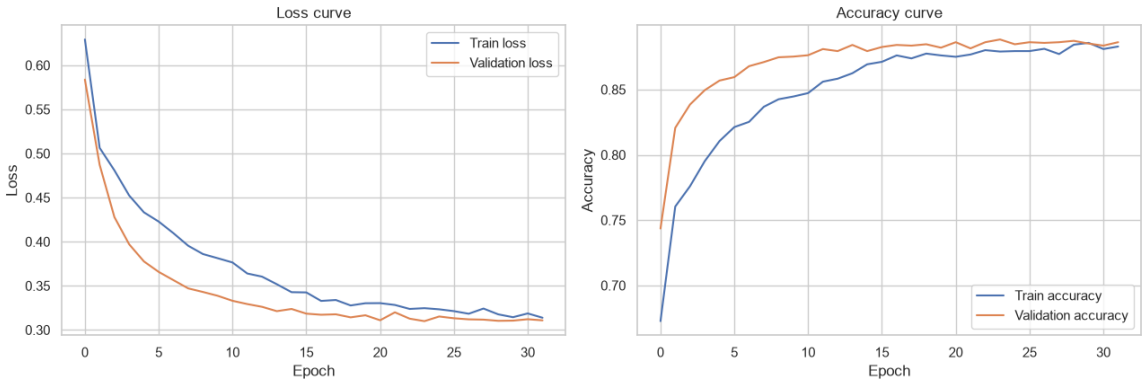
- **ReLU activation (hidden layers):** computationally cheap and avoids the vanishing-gradient problem that affects saturating activations like sigmoid/tanh in deeper networks, allowing the network to learn faster.
- **Sigmoid activation (output layer):** squashes the final score into a probability in [0, 1], which is the correct output form for binary classification and lets the decision threshold (0.5 by default) be tuned if the operational cost of false negatives vs false positives changes.
- **Batch Normalisation**: normalises the activations flowing between layers, which stabilises and speeds up training and provides a mild regularising effect.
- **Dropout (0.3, then 0.2**): randomly disables a fraction of units on each training step, forcing the network to not rely on any single feature combination too heavily — this directly targets overfitting, which is a real risk given the dataset's moderate size relative to the network's parameter count.
- **Binary cross-entropy loss**: the standard loss function for binary classification; it penalises confident wrong predictions much more heavily than uncertain ones, which is a good property for a security alerting system.
- **Adam optimiser**: adapts the learning rate per parameter during training, converging faster and more reliably than plain stochastic gradient descent on this kind of tabular problem.
- **Class weighting**: attack_detected is imbalanced (attacks are the minority class). Rather than leaving the model free to maximise raw accuracy by mostly predicting 'normal', class weights inflate the loss contribution of the minority (attack) class so the model is pushed to actually learn to recognise attacks.
- **Patience-based early stopping (patience = 8, monitoring validation loss, restoring the best weights):** training stops once validation loss has not improved for 8 consecutive epochs, which prevents the network from memorising the training set at the expense of generalisation — this directly satisfies the overfitting-prevention requirement of the project brief.

3. Model Result Analysis

Test-set metrics from the completed run:

Metric	**Value**	**Metric**	**Value**
Accuracy	*0.8941*	F1-score	*0.8676*
Precision	*0.9837*	AUC-ROC	*0.888*
Recall	*0.7761*	Test loss	0.3099

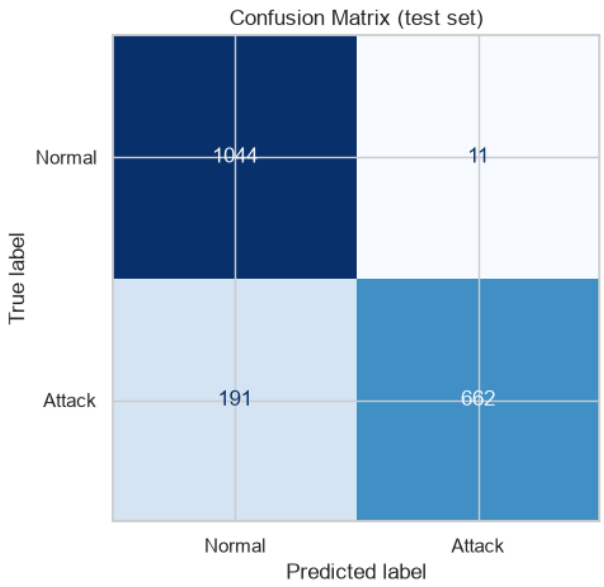
3.1 Reading the training curves



The loss curve (train vs validation loss per epoch) shows whether the model is still learning or has started to overfit: if training loss keeps falling while validation loss flattens or rises, that is the signature of overfitting, and it is exactly what early stopping is designed to catch. Early stopping did trigger: training ran for 32 epochs (of a maximum 100) before the patience-8 monitor on validation loss halted it, and the best weights were restored from epoch 24 (validation loss 0.3099). Validation loss tracked training loss closely throughout — it actually sat slightly below the training curve for most of training, a typical effect of dropout being active only in the training pass — and both curves flattened out from roughly epoch 15 onward in the 0.31–0.32 range with no divergence, so there is no sign of overfitting before the stop.

The accuracy curve tells a similar story for the accuracy metric, but on an imbalanced dataset like this one, accuracy on its own is a weak signal — a model that always predicts 'normal' can still score a high accuracy while catching zero attacks. This is why precision, recall, F1 and AUC-ROC are tracked as well.

3.2 Precision, recall, F1 and the confusion matrix

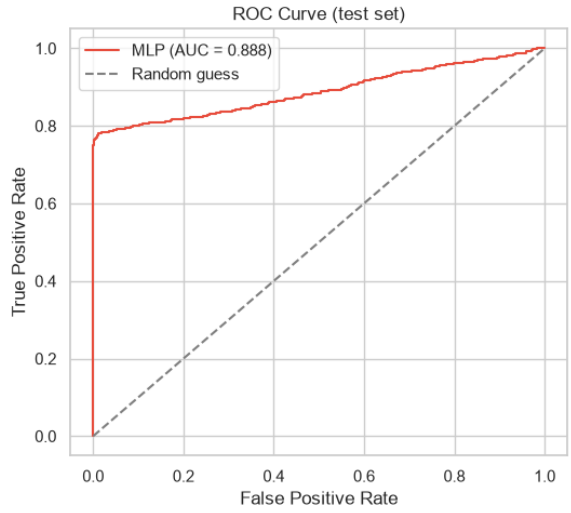


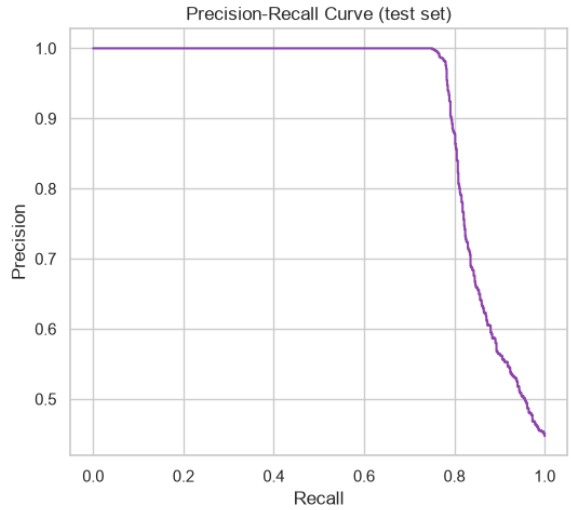
In an intrusion-detection setting, precision and recall trade off two different costs. A false negative (an actual attack scored as 'normal') means a real intrusion slips through undetected — often the more expensive mistake in security. A false positive (a normal session flagged as an attack) wastes analyst time investigating a non-issue, and too many of

these cause alert fatigue, where analysts start ignoring alerts altogether. F1-score balances these two concerns into a single number, which is why it is logged every epoch (via the custom callback in the notebook) alongside the built-in Keras metrics.

The test-set confusion matrix shows 662 of the 853 attack sessions correctly caught (true positives) and 191 missed (false negatives), for a recall of 77.6%. Only 11 of the 1,055 normal sessions were incorrectly flagged (false positives), giving a very high precision of 98.4%. This is a precision-heavy trade-off: the model rarely bothers an analyst with a false alarm, but misses roughly one in five real attacks. That balance is only partially acceptable for a real deployment — the low false-positive rate avoids alert fatigue, but a 22% miss rate on genuine attacks is a meaningful security gap. Lowering the 0.5 decision threshold, or increasing the weight on the minority (attack) class beyond the current ~1.12, would likely convert some of these false negatives into true positives at the cost of a few more false positives.

3.3 ROC curve, PR curve and AUC-ROC





The ROC curve plots the true-positive rate against the false-positive rate across all possible decision thresholds; AUC-ROC summarises this into one number, where 0.5 is no better than random guessing and 1.0 is a perfect classifier. Because attack_detected is imbalanced, the Precision-Recall curve is often a more informative view than the ROC curve alone, since it focuses specifically on how well the model performs on the minority (attack) class rather than being dominated by the easy majority class.

AUC-ROC on the test set is 0.888, clearly and substantially better than random guessing (0.5). The ROC curve rises steeply near the origin, confirming a high true-positive rate is reachable at a very low false-positive rate. The Precision-Recall curve stays at or near 1.0 up to roughly 75–78% recall, then drops off sharply — precision falls below 0.5 as recall approaches 1.0. In other words, the model catches the 'easy' majority of attacks with almost no false alarms, but the remaining attacks are much harder to separate from normal traffic, and pushing for higher recall quickly costs precision.

3.4 Overall effectiveness

Overall, the model is reasonably effective as a first-pass intrusion-detection filter but not yet strong enough to stand alone. The headline numbers — 89.4% accuracy, 98.4% precision, 77.6% recall, 86.8% F1 and 0.888 AUC-ROC — show that class imbalance was handled well: precision, recall, F1 and AUC-ROC were tracked rather than accuracy alone, and class weighting during training pushed the model to actually learn the minority (attack) class instead of defaulting to 'normal'. In this run the SHAP explainability step (notebook §8) did not execute — the shap package was not installed in the kernel that produced these results, so no feature-attribution plot was generated — which should be rerun before submission if a feature-importance discussion is required. The clearest limitation is recall: the model misses roughly one in five real attacks, and the PR curve shows precision degrading quickly past ~75% recall, so threshold tuning, heavier class weighting, or techniques like SMOTE are worth exploring if catching more attacks matters more than minimising analyst alerts.

4. Real-World Use of Models in Cybersecurity

Machine learning and deep learning models are used across several areas of practical cybersecurity beyond the single scenario built in this project:

- Intrusion detection: classifying network sessions or flows as benign or malicious in real time, as implemented here, often as a complement to signature-based systems that can only catch known attack patterns.
- Threat classification: once malicious activity is flagged, a secondary model can categorise the type of threat (e.g. brute-force, DDoS, malware beaconing), helping responders prioritise and route incidents correctly.
- Anomaly detection: unsupervised or semi-supervised models (autoencoders, clustering) flag sessions or behaviour that deviate from a learned baseline of 'normal', which is useful for catching novel attacks that have no prior labelled examples.
- Risk assessment: models can score entities (users, hosts, IP addresses) on an ongoing basis, feeding into a broader risk-scoring system that security teams use to prioritise investigation and access decisions.

4.1 Strengths in practical deployments

ML/DL-based detectors can generalise to attack variants that were never seen during training, something purely signature-based systems cannot do. They also scale well to the volume of traffic a large organisation generates, where manual review of every session or flow is not feasible. Once trained and deployed as a service (as this project does with FastAPI), a model can score sessions in milliseconds, enabling real-time response rather than after-the-fact forensic analysis.

4.2 Limitations and common challenges

- Data quality: models are only as good as the data used to train them. Mislabelled sessions, missing fields, or a dataset that does not represent the organisation's actual traffic patterns will directly degrade real-world performance.
- Concept drift: attacker behaviour and normal user behaviour both change over time (new attack tools, new applications, new usage patterns), so a model trained once will gradually become stale unless it is periodically retrained on fresh data and its performance is continuously monitored.
- False positives (and alert fatigue): even a model with strong aggregate metrics can generate enough false alarms in absolute terms, given the huge volume of legitimate traffic in a real network, to overwhelm a security team — this is why precision, not just recall, matters operationally.
- Interpretability: security analysts and incident responders need to understand why a session was flagged before they can act on it or defend the decision (e.g. in a compliance audit). This project addresses that limitation directly by including a SHAP-based explainability step (notebook §8), which attributes each prediction to the specific input features that drove it.
- Adversarial evasion: attackers who know (or can infer) that a model is in use can deliberately craft traffic to stay under its decision threshold, which is a known open problem for ML-based security systems generally.

4.3 References

Sommer, R., & Paxson, V. (2010). *Outside the closed world: On using machine learning for network intrusion detection*. In Proceedings of the 2010 IEEE Symposium on Security and Privacy (pp. 305-316). IEEE.

Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection*. IEEE Communications Surveys & Tutorials*. 18(2):1153-1176.

Apruzzese, G., Colajanni, M., Ferretti, L., Guido, A., & Marchetti, M. (2018). *On the effectiveness of machine and deep learning for cyber security*. In 2018 10th International Conference on Cyber Conflict (CyCon) (pp. 371-390). IEEE.

Ahmad, Z., Shahid Khan, A., Wai Shiang, C., Abdullah, J., & Ahmad, F. (2021). Network intrusion detection system: A systematic study of machine learning and deep learning approaches*. Transactions on Emerging Telecommunications Technologies.* 32(1), e4150.

Lundberg, S. M., & Lee, S.-I. (2017). *A unified approach to interpreting model predictions*. In Advances in Neural Information Processing Systems 30 (NeurIPS 2017).